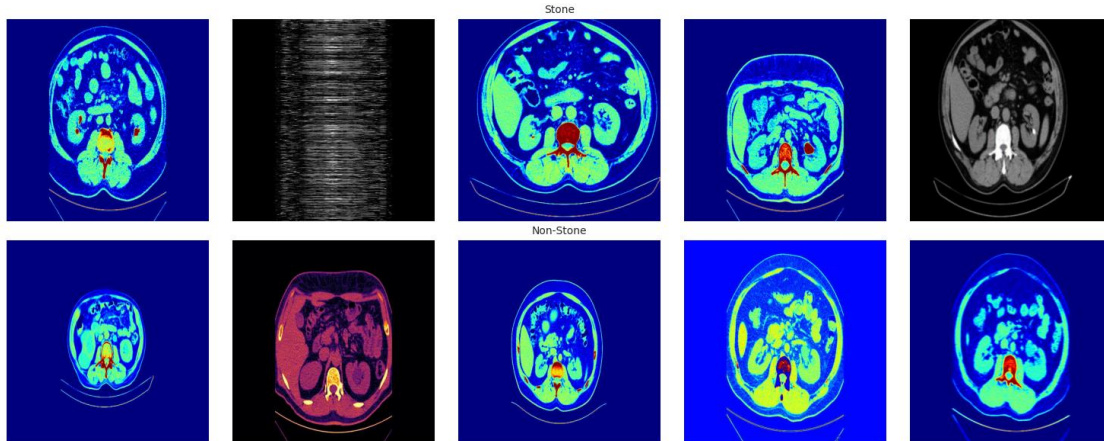


# □ Kidney Stone Classification using X-Distill

## (ViT → CNN Knowledge Distillation)



```
import numpy as np
import pandas as pd
import os

root_path = '/kaggle/input/kidney-stone-axial-ct-imaging-colored-
mixeddata/Axial CT Imaging Dataset for AI-Powered Kidney Stone Detection A
Resource for Deep Learning Research dataset/train/'

image_paths = []
labels = []

for label in os.listdir(root_path):
    label_path = os.path.join(root_path, label)
    if os.path.isdir(label_path):
        for img_file in os.listdir(label_path):
            if img_file.lower().endswith(('.png', '.jpg', '.jpeg')):
                image_paths.append(os.path.join(label_path, img_file))
                labels.append(label)

df = pd.DataFrame({'image_path': image_paths, 'label': labels})

df.head()

          image_path  label
0  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Stone
1  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Stone
2  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Stone
3  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Stone
4  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Stone

df.tail()

          image_path  label
1995  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Non-Stone
```

```

1996 /kaggle/input/kidney-stone-axial-ct-imaging-co... Non-Stone
1997 /kaggle/input/kidney-stone-axial-ct-imaging-co... Non-Stone
1998 /kaggle/input/kidney-stone-axial-ct-imaging-co... Non-Stone
1999 /kaggle/input/kidney-stone-axial-ct-imaging-co... Non-Stone

df.shape

(2000, 2)

df.columns

Index(['image_path', 'label'], dtype='object')

df.duplicated().sum()

0

df.isnull().sum()

image_path    0
label         0
dtype: int64

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   image_path  2000 non-null   object
 1   label       2000 non-null   object
dtypes: object(2)
memory usage: 31.4+ KB

df['label'].unique()

array(['Stone', 'Non-Stone'], dtype=object)

df['label'].value_counts()

label
Stone      1000
Non-Stone   1000
Name: count, dtype: int64

import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style("whitegrid")

fig, ax = plt.subplots(figsize=(8, 6))
sns.countplot(data=df, x="label", palette="viridis", ax=ax)

ax.set_title("Distribution of Stone Types", fontsize=14, fontweight='bold')
ax.set_xlabel("Tumor Type", fontsize=12)

```

```

ax.set_ylabel("Count", fontsize=12)

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=11, color='black',
                xytext=(0, 5), textcoords='offset points')

plt.show()

label_counts = df["label"].value_counts()

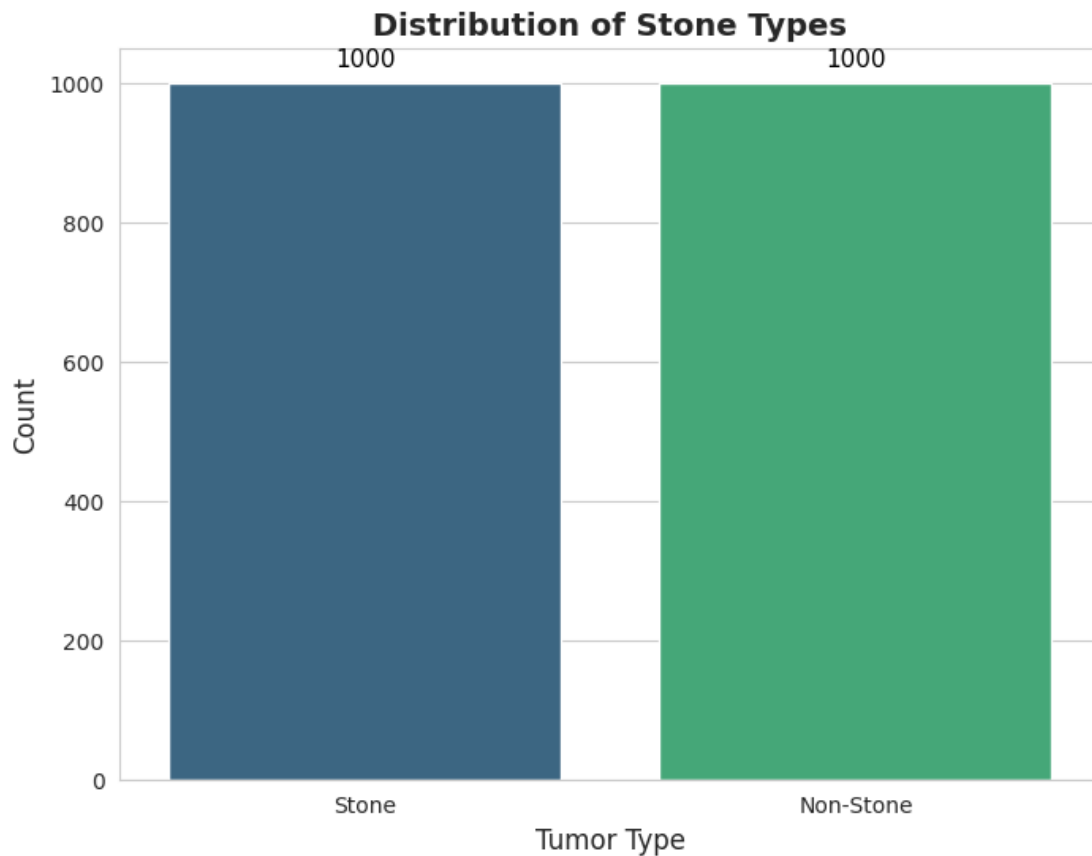
fig, ax = plt.subplots(figsize=(10, 8))
colors = sns.color_palette("viridis", len(label_counts))

ax.pie(label_counts, labels=label_counts.index, autopct='%1.1f%%',
        startangle=140, colors=colors, textprops={'fontsize': 8, 'weight':
        'bold'},
        wedgeprops={'edgecolor': 'black', 'linewidth': 1})

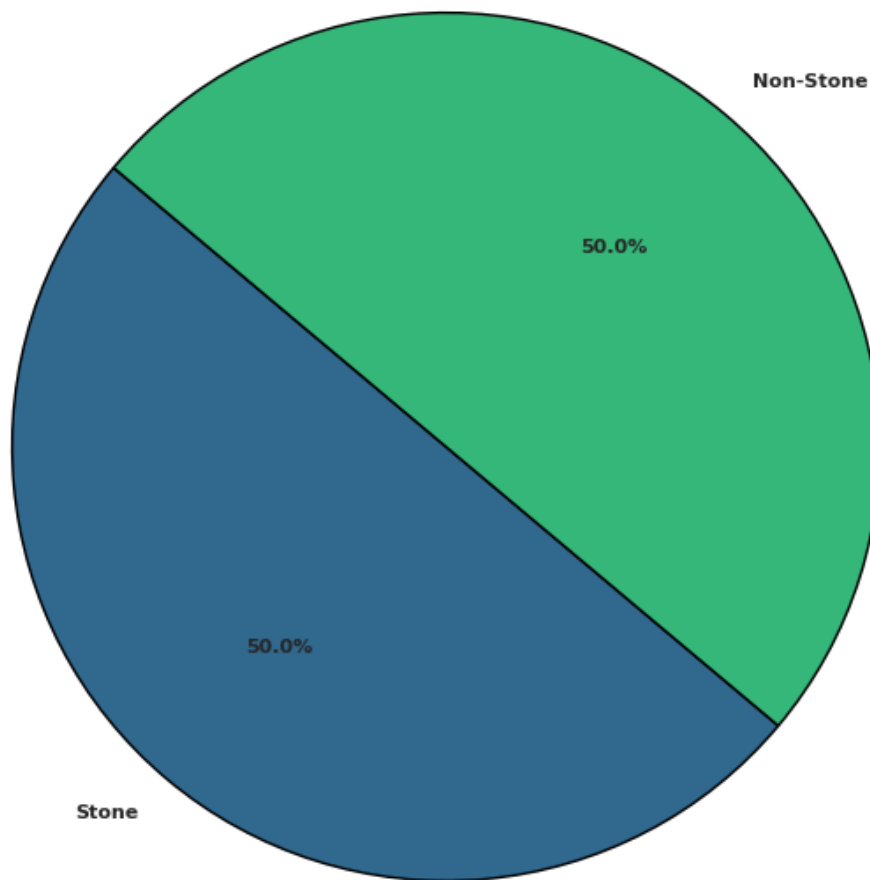
ax.set_title("Distribution of Stone Types - Pie Chart", fontsize=14,
fontweight='bold')

plt.show()

```



## Distribution of Stone Types - Pie Chart



```
from PIL import Image

num_images = 5

unique_labels = df['label'].unique()

plt.figure(figsize=(15, len(unique_labels) * 3))

for row_idx, label in enumerate(unique_labels):

    label_images = df[df['label'] ==
label].head(num_images)['image_path'].tolist()

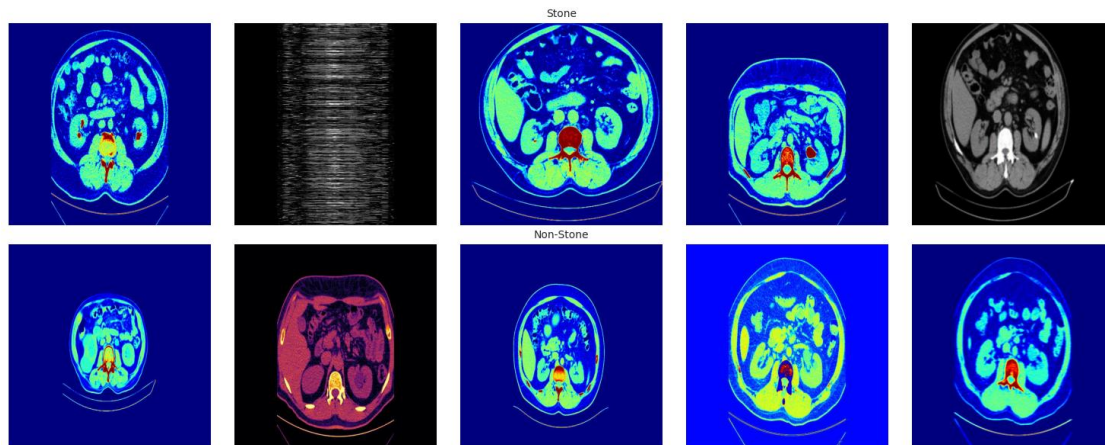
    for col_idx, img_path in enumerate(label_images):
        plt_idx = row_idx * num_images + col_idx + 1
        plt.subplot(len(unique_labels), num_images, plt_idx)
        img = Image.open(img_path)
        plt.imshow(img)
```

```

plt.axis('off')
if col_idx == 2:
    plt.title(label, fontsize=10)

plt.tight_layout()
plt.show()

```



df

|      | image_path                                        | label     |
|------|---------------------------------------------------|-----------|
| 0    | /kaggle/input/kidney-stone-axial-ct-imaging-co... | Stone     |
| 1    | /kaggle/input/kidney-stone-axial-ct-imaging-co... | Stone     |
| 2    | /kaggle/input/kidney-stone-axial-ct-imaging-co... | Stone     |
| 3    | /kaggle/input/kidney-stone-axial-ct-imaging-co... | Stone     |
| 4    | /kaggle/input/kidney-stone-axial-ct-imaging-co... | Stone     |
| ...  | ...                                               | ...       |
| 1995 | /kaggle/input/kidney-stone-axial-ct-imaging-co... | Non-Stone |
| 1996 | /kaggle/input/kidney-stone-axial-ct-imaging-co... | Non-Stone |
| 1997 | /kaggle/input/kidney-stone-axial-ct-imaging-co... | Non-Stone |
| 1998 | /kaggle/input/kidney-stone-axial-ct-imaging-co... | Non-Stone |
| 1999 | /kaggle/input/kidney-stone-axial-ct-imaging-co... | Non-Stone |

[2000 rows x 2 columns]

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import models, transforms
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
from PIL import Image

```

```
df['label'] = df['label'].map({'Stone': 1, 'Non-Stone': 0})
```

```

train_df, test_df = train_test_split(df, test_size=0.2, stratify=df['label'],
random_state=42)
train_df, val_df = train_test_split(train_df, test_size=0.2,
stratify=train_df['label'], random_state=42)

```

```

class KidneyDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df
        self.transform = transform
    def __len__(self):
        return len(self.df)
    def __getitem__(self, idx):
        img_path = self.df.iloc[idx]['image_path']
        label = self.df.iloc[idx]['label']
        image = Image.open(img_path).convert('RGB')
        if self.transform:
            image = self.transform(image)
        return image, label

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

train_loader = DataLoader(KidneyDataset(train_df, transform), batch_size=32,
shuffle=True)
val_loader = DataLoader(KidneyDataset(val_df, transform), batch_size=32,
shuffle=False)
test_loader = DataLoader(KidneyDataset(test_df, transform), batch_size=32,
shuffle=False)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

teacher = torch.hub.load('facebookresearch/dinov2', 'dinov2_vitl14')
teacher = teacher.to(device)
teacher.eval()

student = models.resnet18(weights=None)
student.fc = nn.Linear(student.fc.in_features, 1024)
student = student.to(device)

distill_criterion = nn.MSELoss()
optimizer = optim.Adam(student.parameters(), lr=0.001)

print("--- Starting Stage 1: X-Distill Feature Matching ---")

for epoch in range(10):
    student.train()
    running_loss = 0.0
    for images, _ in train_loader:
        images = images.to(device)
        with torch.no_grad():
            t_features = teacher(images)
            s_features = student(images)

```

```

        loss = distill_criterion(s_features, t_features)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    print(f'Distill Epoch {epoch+1}, Loss:
    {running_loss/len(train_loader):.4f}')

student.fc = nn.Linear(512, 2)
student = student.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(student.parameters(), lr=0.0001)

train_losses, val_losses, train_accs, val_accs = [], [], [], []

print("\n--- Starting Stage 2: Fine-Tuning ---")

for epoch in range(10):
    student.train()
    t_loss, t_correct = 0.0, 0

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)
        outputs = student(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        t_loss += loss.item() * images.size(0)
        t_correct += (outputs.argmax(1) == labels).sum().item()

    student.eval()
    v_loss, v_correct = 0.0, 0

    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device)
            outputs = student(images)
            loss = criterion(outputs, labels)
            v_loss += loss.item() * images.size(0)
            v_correct += (outputs.argmax(1) == labels).sum().item()

    train_loss = t_loss / len(train_df)
    val_loss    = v_loss / len(val_df)
    train_acc   = t_correct / len(train_df)
    val_acc     = v_correct / len(val_df)

    train_losses.append(train_loss)
    val_losses.append(val_loss)

```

```

train_accs.append(train_acc)
val_accs.append(val_acc)

print(f'FT Epoch {epoch+1}: Train Loss: {train_loss:.4f} Acc: {train_acc:.4f} Val Loss: {val_loss:.4f} Acc: {val_acc:.4f}')

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(train_losses, label='Train Loss')
ax[0].plot(val_losses, label='Val Loss')
ax[0].set_title('Loss Curves')
ax[0].legend()
ax[1].plot(train_accs, label='Train Acc')
ax[1].plot(val_accs, label='Val Acc')
ax[1].set_title('Accuracy Curves')
ax[1].legend()
plt.show()

student.eval()
y_true, y_pred = [], []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        outputs = student(images)
        preds = outputs.argmax(1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(preds.cpu().numpy())

print("\nClassification Report:")
print(classification_report(y_true, y_pred, target_names=['Non-Stone', 'Stone'], digits=4))

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Non-Stone', 'Stone'],
            yticklabels=['Non-Stone', 'Stone'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Test Confusion Matrix')
plt.show()

Downloading: "https://github.com/facebookresearch/dinov2/zipball/main" to
/root/.cache/torch/hub/main.zip
/root/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/swiglu_ffn.
py:51: UserWarning: xFormers is not available (SwiGLU)
  warnings.warn("xFormers is not available (SwiGLU)")
/root/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/attention.p
y:33: UserWarning: xFormers is not available (Attention)
  warnings.warn("xFormers is not available (Attention)")
/root/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/block.py:40
: UserWarning: xFormers is not available (Block)

```



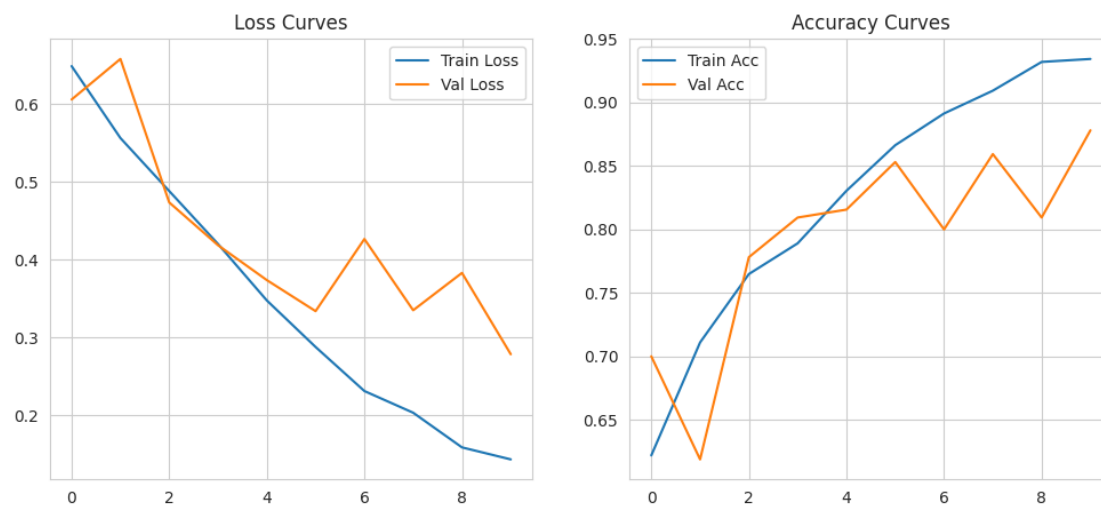
```
warnings.warn("xFormers is not available (Block)")
Downloading:
"https://dl.fbaipublicfiles.com/dinov2/dinov2_vitl14/dinov2_vitl14_pretrain.p
th" to /root/.cache/torch/hub/checkpoints/dinov2_vitl14_pretrain.pth
100%|██████████| 1.13G/1.13G [00:13<00:00, 91.1MB/s]
```

--- Starting Stage 1: X-Distill Feature Matching ---

```
Distill Epoch 1, Loss: 0.6453
Distill Epoch 2, Loss: 0.4513
Distill Epoch 3, Loss: 0.4338
Distill Epoch 4, Loss: 0.4027
Distill Epoch 5, Loss: 0.3900
Distill Epoch 6, Loss: 0.3652
Distill Epoch 7, Loss: 0.3574
Distill Epoch 8, Loss: 0.3446
Distill Epoch 9, Loss: 0.3204
Distill Epoch 10, Loss: 0.2988
```

--- Starting Stage 2: Fine-Tuning ---

```
FT Epoch 1:  Train Loss: 0.6483  Acc: 0.6219  Val Loss: 0.6055  Acc: 0.7000
FT Epoch 2:  Train Loss: 0.5561  Acc: 0.7109  Val Loss: 0.6575  Acc: 0.6188
FT Epoch 3:  Train Loss: 0.4880  Acc: 0.7648  Val Loss: 0.4734  Acc: 0.7781
FT Epoch 4:  Train Loss: 0.4201  Acc: 0.7891  Val Loss: 0.4185  Acc: 0.8094
FT Epoch 5:  Train Loss: 0.3477  Acc: 0.8305  Val Loss: 0.3738  Acc: 0.8156
FT Epoch 6:  Train Loss: 0.2879  Acc: 0.8664  Val Loss: 0.3339  Acc: 0.8531
FT Epoch 7:  Train Loss: 0.2314  Acc: 0.8914  Val Loss: 0.4265  Acc: 0.8000
FT Epoch 8:  Train Loss: 0.2036  Acc: 0.9094  Val Loss: 0.3351  Acc: 0.8594
FT Epoch 9:  Train Loss: 0.1591  Acc: 0.9320  Val Loss: 0.3830  Acc: 0.8094
FT Epoch 10: Train Loss: 0.1437  Acc: 0.9344  Val Loss: 0.2786  Acc: 0.8781
```



Classification Report:

|           | precision | recall | f1-score | support |
|-----------|-----------|--------|----------|---------|
| Non-Stone | 0.8498    | 0.9050 | 0.8765   | 200     |
| Stone     | 0.8984    | 0.8400 | 0.8682   | 200     |

|              |        |        |        |     |
|--------------|--------|--------|--------|-----|
| accuracy     |        |        | 0.8725 | 400 |
| macro avg    | 0.8741 | 0.8725 | 0.8724 | 400 |
| weighted avg | 0.8741 | 0.8725 | 0.8724 | 400 |

